

升级（Trump）纸牌游戏 — 完整规则与AI策略文档

代码仓库: card-game-server-upstream (backend-decoupling分支) 文档版本: v2.0
| 更新日期: 2026-06-10

目录

- [1. 游戏概述](#)
- [2. 牌面体系](#)
- [3. 游戏流程详解](#)
- [4. 出牌规则与跟牌约束](#)
- [5. 轮次结算与扣底](#)
- [6. 游戏结束与升级规则](#)
- [7. 系统架构](#)
- [8. 规则AI策略 \(v28\)](#)
- [9. DMC V9 AI策略](#)
- [10. AI决策优先级与Fallback机制](#)

1. 游戏概述

升级（又称"拖拉机"）是一款4人扑克牌游戏，使用两副标准扑克牌（共108张）。

1.1 基本设定

项目	说明
玩家人数	4人
组队方式	固定搭档：0号与2号一队，1号与3号一队
牌组	两副扑克（ $54 \times 2 = 108$ 张）
花色	♠黑桃(a)、♥红桃(b)、♣梅花(c)、♦方块(d)、★王(z)
起始级别	从A开始，按 A→2→3→4→5→6→7→8→9→10→J→Q→K→A 循环
	底牌

1.2 胜负判定

- 庄家队：需要阻止闲家队拿到足够分数
- 闲家队：需要收集尽可能多的分数牌来夺庄

分牌与分值：

牌面	分值
5	5分
10	10分
K	10分

一副牌总分 = 4×5 + 4×10 + 4×10 = 100分，两副牌总分 = 200分

2. 牌面体系

2.1 牌的表示

每张牌用 `card_type` 字符串表示，格式为 {名称}-{rank}-{花色}：

示例： "A-13-a" = 黑桃A， "5-4-b" = 红桃5， "w-14-z" = 小王， "W-15-z" = 大王

rank映射：

名称	2	3	4	5	6	7	8	9	10	J	Q	K	A	小王	大王
rank	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

2.2 主牌判定

一张牌是主牌的条件（满足任一即为真）：

1. 大小王 → 永远是主牌
2. 级牌 → 名字等于当前级别的牌（如打A时所有A都是主牌）
3. 固定主牌 → 名字为2、3、5的牌（ `FIXED_ZHU_NAMES = {'2', '3', '5'}` ）
4. 主花色牌 → 花色等于亮主确定的主花色

判定优先级：大王 > 小王 > 主花色级牌 > 其他级牌 > 主花色固定主牌 > 其他固定主牌 > 主花色普通牌

2.3 主牌排序 (get_zhu_rank)

主牌内部有严格的排序规则：

大王(W) > 小王(w) > 主花色级牌 > 其他花色级牌 > 主花色5 > 其他花色5 > 主花色3 > 其他花色3 > 主花色2 > 其他花色2

注意：当级牌本身就是2、3或5时，该牌按级牌排序（排在固定主之上），不再按固定主排序。

2.4 副牌排序

同一花色的副牌按rank排序：A(13) > K(12) > Q(11) > J(10) > 10(9) > 9(8) > 8(7) > 7(6) > 6(5) > 4(3)

注意：副牌中不包含级牌、2、3、5——这些牌都是主牌，已被归入主牌排序。所以副牌序列中排除了这些rank。

3. 游戏流程详解

3.1 整体流程

等待入座 → 亮主 → 吃牌 → 扣牌 → 拾牌/换牌 → 出牌(多轮) → 游戏结束
WAITING LIANGZHU CHIPAI KOUPAI HUANPAI PLAYING GAME_OVER

3.2 阶段1：等待入座 (WAITING)

- 4个座位 (0-3号)，玩家通过WebSocket加入并选座
- 座位0与2为庄家队，1与3为闲家队
- 4人入座后可开始游戏 (start_game)
- 也支持"一键机器人"模式 (/api/start-robot-game)：4个位置自动填入机器人

3.3 阶段2：亮主 (LIANGZHU)

亮主是决定主花色的关键阶段。

3.3.1 发牌

- 从108张牌中洗牌
- 每人发26张 (共104张)，剩余4张为底牌
- 发牌后进入亮主阶段

3.3.2 亮主牌型

亮主需要出特定牌组合来声明主花色，牌型越强优先级越高：

牌型	中文名	优先级排名	组合方式	是否需要王
duolian	多连对	1（最强）	同花色6张以上连对（3连对+）	不需要
threeking	三王	2	任意3张王 + 手中需有一个对子或连对	本身就是3张王
shuanglian	双连对	3	同花色4张连对(2连对) + 1王	需要1张王
danlian	单对子	4（最弱）	同花色1对 + 1王	需要1张王

亮主规则： - 多连对（≥3连对）：不需要王即可亮主，纯副牌连对即可 - 其余牌型（三王、双连对、单对子）：前提都是手中需要有王 - 三王牌型：需要3个王且手中有至少一个对子或连对，对子/连对的花色确定为主花色 - 同队不能重复亮主（队友已亮，自己自动跳过） - 异队可以吃牌（chipai）来争夺亮主权 - 亮主后确定主花色 = 亮主牌的花色

3.3.3 无人亮主

如果4人全部选择不亮主（pass_liangzhu）： - 执行 handle_no_liang() - 随机翻一张底牌确定主花色 - 直接跳到KOUPAI阶段（无吃牌环节）

3.4 阶段3：吃牌（CHIPAI）

亮主后，异队玩家可以选择"吃牌"来争夺亮主权。

吃牌规则

- 1. 只有异队（与亮主者不同队）可以吃牌
- 2. 吃牌者需要出合法的亮主牌型
- 3. 吃牌成功后，亮主权归吃牌者
- 4. 异队两人都不吃 → 亮主权归原亮主者
- 5. 先到先得（手牌最强的先决定）

吃牌结果： - 有人吃牌 → 吃牌返牌后进入KOUPAI（扣牌）阶段 - 无人吃牌 → 直接进入KOUPAI（扣牌）阶段

吃牌返牌规则

吃牌成功后，吃牌者（赢家）获得被吃者的亮主牌，同时需要返牌给被吃者：

1. 返固定主牌：吃的牌中有N张王牌 → 返N张固定主牌（2、3、5、级牌、王中任选），不可返亮主花色的主牌（如亮红桃主，不可返红桃K等）
2. 返同花色副牌：吃的牌中有M张副牌（对或连对）→ 返M张同花色副牌（不要求同rank）
3. 返牌总数 = 被吃的牌数
4. 返牌策略：在不破坏手中对子/连对等牌型的基础上，优先返最小的牌
5. 返牌不影响吃牌结果——即使全部返回去，吃牌仍然成功

示例：第一个亮主者亮了1张王+1对红桃K，被另一个用1张王+1对黑桃A吃掉。吃牌者需要返还：
- 1张固定主牌（2/3/5/级牌/王中选一张最小的，不可返红桃主牌） - 2张红桃副牌（在不破坏对子/连对基础上选最小的红桃副牌）

3.5 阶段4：扣牌（KOU PAI）

扣牌是庄家的关键决策——扣牌者拾起底牌后，从手牌中选4张扣回底牌。

扣牌规则

1. 扣牌者：庄家队中级别较高的一人（`_get_koupai_picker()`）
2. 扣牌者先拾起4张底牌加入手牌，再从手牌中选择4张扣回底牌
3. 底牌中的分数牌在游戏结束时的扣底环节会翻倍计算

扣牌后

扣牌完成后： - 如果扣回的牌中有亮主花色的牌 → 进入HUANPAI（换牌）阶段 - 如果扣回的牌中没有亮主花色的牌 → 直接进入PLAYING阶段

3.6 阶段5：换牌（HUAN PAI）

扣牌后，如果扣回的牌中包含亮主花色的牌，亮主者需要与扣牌者进行换牌。

换牌规则

1. 触发条件：扣牌者扣回的牌中有亮主花色的牌
2. 亮主者看到扣牌中亮主花色的牌，可以选择捡起这些牌
3. 如果接受，需返出同数量的非亮主花色牌
4. 换牌完成后进入PLAYING阶段

3.7 阶段6：出牌（PLAYING）

这是游戏的核心阶段，4人轮流出牌，每轮决出胜负。

出牌顺序

- 1. 每局由庄家先出牌
- 2. 首出者（第一个出牌的人）可以出任意合法牌型
- 3. 后续3人必须按照跟牌规则出牌
- 4. 一轮4人出完后结算，赢家成为下一轮的首出者
- 5. 重复直到所有人手牌出完

首出者规则

首出者可以出的牌型：

牌型	代码标识	说明
副单	fudan	一张非主牌
副对	fudui	同花色同rank的两张副牌
副连对	fulian	同花色连续rank的对子（2对=双连对，3对+ =多连对）
主单	zhudan	一张主牌
主对	zhudui	同rank的两张主牌
主连对	zhulian	连续rank的对子（2对=双连对，3对+ =多连对）

连对分类： - 双连对：同花色连续rank的2对牌（如33-44、KK-AA），代码标识 fulian2/zhulian2
- 多连对：同花色连续rank的3对及以上牌（如33-44-55、KK-AA-22），代码标识 fulian3/zhulian3 等

3.8 游戏结束（GAME_OVER）

所有手牌出完后，进行最终结算。

4. 出牌规则与跟牌约束

跟牌是升级游戏最核心的规则约束，代码实现在 `_validate_play()` 方法中。

4.1 首出主牌时的跟牌规则

当首出者出主牌时：

1. 必须跟主牌：手中有主牌必须出主牌，不能出副牌
2. 主牌数量约束：必须出与首出相同数量的主牌
3. 主牌不够：手中主牌不足以凑齐数量时，出全部主牌 + 最小副牌补齐
4. 无主牌：手中完全无主牌时，可以出任意副牌

主对的特殊规则：- 首出主对时，如果有主牌但凑不出主对，必须出最大的主牌（不能出小主牌留大的）

主连对的特殊规则：- 跟主连对时，有主牌必须出所有主牌，不够n张用副牌补齐 - 只有手牌0张主牌时，才可纯用副牌凑齐 - 选项生成独立分支：先找主连对 → 有主对出主对 → 有主散牌凑主牌 → 0张主牌用副牌补

4.2 首出副牌时的跟牌规则

当首出者出副牌时：

规则1：有同花色副牌必须出

- 手中有与首出同花色的副牌时，必须优先出同花色副牌
- 出的同花色副牌数量 = $\min(\text{手中同花色副牌数}, \text{首出牌数})$

规则2：不可以用主牌替代副牌

- 在有同花色副牌可出的前提下，不能用主牌替代同花色副牌
- 例：首出 ♠3，手中有 ♠5 和主牌，必须出 ♠5，不能出主牌

规则3：同花色不够时可以补主牌

- 同花色副牌数量不足时，可以用主牌补齐
- 例：首出2张 ♠，手中只有1张 ♠副牌，出1张 ♠ + 1张主牌

规则4：绝门时出任意牌

- 完全没有该花色的牌时（绝门，手中同花色副牌数=0），可以出任意牌
- 绝门判定：`hand_color_count == 0`（手中该花色非主牌为0张）
- 绝门出主牌 = "毙牌"，可以赢副牌

规则5：跟副对/副连对时的特殊规则

- 有同花色对子时：验证层强制必须出对子（有同花色对子必须出对子）
- 1张同花色时：补牌无强制，副牌和主牌均可补。AI策略优先用其他副牌补
- 绝门(0张同花色)跟副对：无强制规则，可出任意2张。不强制出主对，AI策略决定是否毙牌
- 绝门(0张同花色)跟副连对：无强制规则，可出任意牌。AI策略优先出无分副散牌，有主连对不主动毙牌

4.3 牌型比较规则（compare_outcards）

两副出牌比较大小的核心逻辑：先比较牌型，牌型一致才可比较后续属性。

牌型优先级（从高到低）

连对(lian) > 多对(duiN, 非连对) > 单对(dui) > 对+散牌(duiN_san) > 单张(dan) > 全散牌(san)

核心原则：不同牌型之间，大牌型永远赢小牌型。例如：- 副对 > 主散牌（对子型 > 散牌型，即使主牌更大也不行）- 主连对 > 副多对 - 副对 > 副散牌

同牌型内的比较规则

副牌 vs 副牌（同花色）

- 单牌：rank大的赢
- 对子：必须同rank才算对子，rank大的赢
- 连对：连续rank对子数多的大；数量相同则最高rank大的赢

主牌 vs 副牌（同牌型前提下）

- 牌型相同时，主牌 > 副牌
- 例：首出副对，跟牌出了主对 → 牌型相同（都是对子），主对 > 副对，主对赢
- 例：首出副对，跟牌出了2张主散牌 → 牌型不同（对子 vs 散牌），副对赢（牌型优先）

主牌 vs 主牌

- 按主牌排序（get_zhu_rank）比较：
- 主连对 > 主对 > 主单
- 同类型比较rank

不同花色副牌

- 首出花色的副牌大于其他花色副牌
- 其他花色之间不可比较（均为副牌时只有首出花色有效）

绝门毙牌规则

当跟牌者绝门（无首出花色）时： - 出主对可以赢副对（牌型相同，主>副） - 出主散牌不能赢副对（牌型不同，对子>散牌） - 绝门时不强制出主对或主连对，AI策略决定是否毙牌

4.4 出牌验证与强检验

手牌数量验证（handle_play）

出牌时验证每种牌的出牌数量不超过手牌中的数量：

```
from collections import Counter
hand_counter = Counter(hand_strs)    # 手牌中各牌的数量
play_counter = Counter(card_strs)    # 出牌中各牌的数量
for cs, count in play_counter.items():
    if cs not in hand_counter:        → "出的牌不在手中"
    if count > hand_counter[cs]:      → "出的牌数量超过手中数量"
```

重要：此检查使用Counter计数而非简单的 in 判断，防止出2张同名牌但手中只有1张的情况。

轮后手牌数相等断言

每轮4人出完牌后，强检验所有人手牌数必须相等：

```
hand_counts = [self.players[i].card_count for i in range(4)]
assert len(set(hand_counts)) == 1, '轮后手牌数不等!'
```

出牌数相等验证

验证层（_validate_play）确保：每轮4人出牌数必须与首出者相同（len(played) < n and player.card_count >= n 时拒绝）。

4.5 合法出牌枚举（_enumerate_legal_plays）

引擎提供 _enumerate_legal_plays(player, is_first) 方法枚举所有合法出牌组合：

1. 首出时：枚举所有可能的牌型组合（单/对/连对）
2. 跟牌时：根据首出牌型，枚举所有满足跟牌约束的组合
3. 返回格式：[{'cards': [card_type, ...], 'type': play_type}]

跟主连对的选项生成分支（独立于副牌分支）：1. 找主连对 → 如找到直接返回 2. 有主对但无主连对 → 提供主对选项 3. 只有主散牌 → 凑n张主散牌；不够n张时必须出所有主牌+副牌补齐 4. 完全无主牌(0张) → 用副牌凑齐

5. 轮次结算与扣底

5.1 轮次结算 (_resolve_epoch)

每轮4人出牌完成后：

- 1. 比较所有出牌，找出最大牌
- 2. 确定赢家座位号 (winner_seat)
- 3. 判断赢家队伍：
- 4. 赢家是闲家 → 闲家得分 = 本轮所有分牌总分
- 5. 赢家是庄家 → 闲家不得分
- 6. 记录最后一轮信息 (last_epoch_winner , last_epoch_cards)

5.2 扣底结算

游戏结束时，如果最后一轮闲家赢了：

倍数 = ceil(赢家出牌中主牌张数 / 2)

- 副牌赢最后一轮 → 倍数 = 0 (不触发扣底)
- 赢家出1张主牌 → 倍数 = 1
- 赢家出2张主牌 → 倍数 = 1
- 赢家出3张主牌 → 倍数 = 2
- 以此类推

扣底分数：- 底牌基础分 × 扣底倍数 - 闲家赢最后一轮 → 闲家得分 += 扣底分数 - 庄家赢最后一轮 → 闲家得分 -= 扣底分数（最低0）

6. 游戏结束与升级规则

6.1 结果判定

闲家得分	结果类型	中文名	升级级数	赢家
0分	guangtou	光头	+4级	庄家
1-39分	xiaoguang	小光	+2级	庄家

闲家得分	结果类型	中文名	升级级数	赢家
40-79分	sheng1	庄胜	+1级	庄家
80-119分	duozhuang0	夺庄+0	+0级	闲家
120-159分	duozhuang1	夺庄+1	+1级	闲家
160-199分	duozhuang2	夺庄+2	+2级	闲家
≥200分	quansheng	全胜	+4级	闲家

6.2 升级规则

- 庄家赢：庄家队两人各加对应级数，下一局继续坐庄
- 闲家赢（夺庄）：闲家队两人各加对应级数，下一局闲家变庄家
- 级数循环：超过K后回到A（`player_level > 13` → 减13）

6.3 庄家确定

- 首局：0号和2号为庄家
- 庄家赢：继续坐庄
- 闲家赢（夺庄）：闲家座位成为新庄家
- 例：原庄家[0,2]，闲家[1,3]夺庄 → 新庄家[1,3]

7. 系统架构

7.1 技术栈

组件	技术
Web框架	aiohttp (异步HTTP/WebSocket)
端口	9999
前端	HTML + JavaScript (WebSocket实时通信)
AI-规则	server/ai.py (纯Python规则引擎)
AI-DMC	rl_dmc/dmc_ai.py (PyTorch DMC模型)
牌组	server/card.py (108张牌管理)

7.2 核心文件结构

```
card-game-server-upstream/
├─ app.py                # aiohttp服务入口, WebSocket处理, API路由
├─ server/
│   ├─ game_engine.py    # 核心游戏引擎 (GameRoom类, ~2607行)
│   ├─ ai.py             # 规则AI (AI类, ~1675行)
│   ├─ card.py           # 牌/牌组定义 (Card, Deck类)
│   └─ constants.py      # 常量定义
├─ rl_dmc/
│   ├─ dmc_ai.py         # DMC V9 AI推理引擎
│   ├─ q_network_v2.py   # QNetworkV2网络定义
│   ├─ env.py            # DMC训练环境 (牌编码/动作空间)
│   └─ models_v9/
│       └─ dmc_v9_best.pt # V9最佳模型 (70MB)
├─ rl_shengji/
│   ├─ obs_v11.py        # 764维观测编码
│   └─ env.py            # 训练环境封装
├─ static/              # 前端静态文件
│   ├─ index.html        # 主页面
│   ├─ game.js           # 游戏逻辑 (WebSocket通信)
│   └─ style.css         # 样式
├─ test_dmc_unit.py      # DMC集成单元测试 (25项)
├─ test_dmc_e2e.py       # 端到端测试 (50局)
├─ Dockerfile           # Docker构建文件
├─ requirements.txt      # Python依赖
└─ .dockerignore
```

7.3 WebSocket通信协议

客户端 → 服务器

action	说明	参数
join	加入房间	room_id, name, seat, client_id
select_seat	换座	seat
leave	离开房间	-
start_game	开始游戏	-

action	说明	参数
liangzhu	亮主	cards, color
pass_liangzhu	不亮主	-
koupai	扣牌	cards
shipai	拾牌+还牌	pick_cards, return_cards
play	出牌	cards

服务器 → 客户端

action	说明
joined	加入成功（含seat/state）
state_update	全局状态更新（每次操作后广播）
liangzhu_result	亮主结果
play_result	出牌结果
game_over	游戏结束（含最终比分和升级信息）
error	错误消息

7.4 HTTP API

路由	方法	说明
/	GET	主页面
/api/rooms	GET	列出所有房间
/api/rooms	POST	创建房间
/api/start-robot-game	POST	一键创建4机器人房间并开始
/api/debug/seed-liangzhu	POST	调试用：指定亮主手牌
/static/{path}	GET	静态文件
/ws	GET	WebSocket连接

7.5 机器人游戏循环 (_robot_game_loop)

纯机器人对局的后台运行机制：

```
while phase != GAME_OVER:
    actions = room.auto_play_current_robot()
    if phase == LIANGZHU and all_ready and no_liangzhu:
        room.handle_no_liang()
    await broadcast_room(room)
    await asyncio.sleep(0.5)    # 每0.5秒一步
```

人机混合对局则通过 `process_robots()` 在人类操作后触发机器人自动操作。

8. 规则AI策略 (v28)

规则AI是纯Python实现的基于规则的AI，当前版本为v28（局部优化天花板）。

8.1 AI类结构

```
class AI:
    def __init__(self, player, now_level, now_color, score_koupai)
    def evaluate_hand() -> float           # 手牌评估
    def decide_liangzhu(chipai_mode=False) # 亮主决策
    def decide_shipai(offer, color)        # 拾牌+还牌
    def decide_huanpai(offer)              # 换牌决策
    def decide_koupai(hole_cards)          # 扣牌决策
    def decide_play(epoch_cards, ...)      # 出牌决策
```

8.2 手牌评估 (evaluate_hand)

手牌总分 = 各维度加总：

维度	权重	说明
主牌	每张+3	主牌越多越好
主对	每对+8	主对是可靠的赢轮牌
主连对	每连+15	主连对几乎必赢
副对	每对+4	副对有一定赢轮能力

维度	权重	说明
副连对	每连+8	副连对更强
分牌	5分+1, 10分+2	分牌在手但容易被收
绝门	每门+3	绝门后可毙牌

8.3 亮主策略 (decide_liangzhu)

候选牌型搜索

按优先级搜索可用的亮主牌型：1. 多连对（6张+连对 + 王）→ 优先级1 2. 三王 + 连对 → 优先级2 3. 双连对（4张连对 + 1王）→ 优先级3 4. 单对子（1对 + 1王）→ 优先级4

亮主收益评估

```
effective_score = hand_score + huanpai_bonus + banker_bonus - chipai_risk

huanpai_bonus = 4.0      # 换牌收益
banker_bonus = 3.0       # 坐庄收益
chipai_risk = best_type_rank × 5.0  # 被吃风险（弱牌型风险大）
```

亮主门槛

- 正常模式：effective_score ≥ 30 才亮主
- 吃牌模式（chipai_mode）：effective_score ≥ 18 才亮主（对手已亮，不亮更被动）

选择策略

从所有候选牌型中，选择优先级最高（数值最小）且花色强度最大的牌型亮主。

8.4 扣牌策略 (decide_koupai)

核心思路：优先绝门，次保结构，最后兜底

1. 保护对子和连对：对子和连对中的牌不扣
2. 绝门优先：如果某花色副牌≤4张，全扣实现绝门
3. 双绝门优先：2个绝门比1个绝门战术价值更高
4. 优先扣无分小牌：绝门时先扣无分牌，5分牌可扣，K/10分不清门
5. 兜底：扣弱花色的无分小牌 → 分牌 → 主牌 → 任意牌

8.5 拾牌/换牌策略

拾牌 (decide_shipai) : - 基本总是拾起 (多拿主牌有利) - 评估offer价值: 分牌+5, 大牌+3, 小牌+1 - 价值太低 (< offer数×0.5) 则不拾 - 还牌选择: 同rank中最弱的牌

换牌 (decide_huanpai) : - 基本总是接受 (主牌总比副牌好) - 返牌选择: 非亮主花色、非王牌、同rank中最弱的牌

8.6 出牌策略 (decide_play) ★核心

出牌策略分两大类: 先手出牌 (首出者) 和跟牌 (后续出牌者)。

8.6.1 先手出牌

先手出牌区分庄家和闲家策略:

庄家先手策略 (_first_play_banker)

庄家目标: 控场、拿分、保过庄

优先级	牌型	说明
1	尾局出主	游戏末尾必出主牌争最后一轮
2	主连对	最可靠的先手赢轮 (几乎必赢)
3	主对×2	可靠赢轮
4	级牌主单	级牌主单赢率76%, 优先于普通主单35%
5	最小主单	无级牌时出最小主单
6	清短门	≤5张花色出无分小牌, 创造绝门
7	无对花色小单	无对子花色出小牌试探
8	副小单	最小无分副单
9	弱副连对	赢率不高但比副单强
10	弱副对	最后才出
11	最大主单	兜底争取赢轮

清门细节: - 优先出有无分牌的花色 (不送分) - 只剩分牌时, 5分可送 (绝门价值>5分), K/10分不清门 - 弱花色阈值: 5张以下

闲家先手策略 (_first_play_defender)

闲家目标：抢分、创造毙牌机会

优先级	牌型	说明
1	尾局出主	同庄家
2	主连对/主对	赢率55%+
3	清短门	创造绝门→毙牌拿分
4	级牌主单	赢率76%
5	最小主单	无级牌时
6	副小单	最后手段

清门差异 (vs庄家)： - 闲家清门更积极：主牌≥5张时≤5张花色也清 - 闲家清门时避免有副对的花色 (对子留着跟牌更有用)

8.6.2 跟牌策略 (_follow_play)

跟牌按首出牌型分6种处理：

首出牌型	处理方法
副单(fudan)	_follow_fudan
副对(fudui)	_follow_fudui
副连对(fulian)	_follow_fulian
主单(zhudan)	_follow_zhudan
主对(zhudui)	_follow_zhudui
主连对(zhulian)	_follow_zhulian
散牌/混合	_follow_generic

核心跟牌原则

- 1. 队友大时 → 积极贴分牌 (分牌最安全的去处)
- 2. 对手大时 → 用最小能管住的牌管 (省大牌)
- 3. 管不住时 → 绝不出分牌 (出最小无分牌)
- 4. 绝门毙牌 → 出最小主牌 (省大牌用于后续关键轮)

5. 绝门队友大 → 贴分牌（不是副小牌）
6. 最后一轮+底牌有分 → 不贴分（防扣底）

跟副单详细流程

1. 判断队友是否领先
2. 有同花色时：
 - a. 队友大 → 贴最小分牌（5→10→K），无分牌出最小
 - b. 对手大 → 出最小能管住的牌
 - 轮次有≥5分 → 认真管（从大到小搜索）
 - 轮次<5分 → 随便出最小的（不值得浪费大牌）
 - c. 管不住 → 出最小无分牌
3. 绝门时：
 - a. 队友大 → 贴最大分牌（跨花色）
 - b. 对手大 → 毙牌（出最小主牌）
 - c. 最后一轮+底牌有分 → 不贴分（防扣底）

跟副对详细流程

1. 有同花色对子 → 必须出对子跟牌（验证层强制）
2. 有1张同花色+需补牌：
 - a. 优先用其他副牌补齐（不浪费主牌）
 - b. 副牌不够再用主牌补
 - c. 验证层无强制要求，AI策略决定
3. 绝门(0张同花色)：
 - a. 无强制规则，可出任意2张
 - b. 队友大 → 贴分牌（不主动毙牌浪费主对）
 - c. 对手大+场上有分>10 → 用主对毙牌抢分
 - d. 对手大+场上无分或分少 → 出最小2张（省主对）

跟副连对详细流程

1. 有同花色副牌：
 - a. 有足够同花色连对/对子 → 正常跟牌
 - b. 同花色不够 → 优先补其他副牌
2. 绝门(0张同花色)：
 - a. 无强制规则，可出任意牌
 - b. 优先出无分副散牌（省主牌）
 - c. 有主连对也不主动毙牌（跟副连对时毙牌代价高）

- d. 队友大 → 贴分牌
- e. 对手大+场上有分>10 → 考虑用主连对毙牌

跟主单/主对详细流程

1. 有主牌时：
 - a. 队友大 → 贴分主牌（如5/10/K级牌或固定主）
 - b. 对手大 → 出最小能管住的主牌
 - c. 管不住 → 出最小主牌
2. 无主牌时：
 - a. 队友大 → 贴分副牌
 - b. 对手大 → 出最小副牌
3. 主对特殊：凑不出主对时出最大主牌（不能留小的）

跟主连对详细流程

1. 有主连对 → 正常跟牌
2. 无主连对但有主对 → 出主对凑齐
3. 有主牌(含主散牌) → 必须出所有主牌，不够n张用副牌补齐
4. 完全无主牌(0张) → 用副牌凑齐

关键规则：跟主连对时，有主牌必须出所有主牌（不能只出部分主牌然后用副牌补），只有0张主牌才可纯用副牌。这是为了确保出牌数与首出者相等。

后处理机制

跟牌结果经过两个后处理步骤：

1. `_validate_and_fix_follow`: 验证跟牌合法性
2. 首出副牌时：确保出了足够的同花色副牌（不够时用副牌替换主牌）
3. 首出主牌时：确保出了足够的主牌（不够时用主牌替换副牌）
4. 绝门判定统一为 `hand_color_count == 0`
5. `_ensure_card_count`: 确保出牌数量 = 首出牌数
6. 手牌足够时必须凑齐
7. 首出主牌：优先补主牌再补副牌
8. 首出副牌：优先补同花色 → 其他副牌 → 主牌
9. SafeFallback（安全兜底）：

- 10. 首出时：出最小单牌
- 11. 跟副对/副连对绝门：可出任意牌，优先无分副散牌
- 12. 跟主连对有主牌：必须出所有主牌+副牌补齐
- 13. 跟主连对0张主牌：用副牌凑齐

9. DMC V9 AI策略

DMC (Deep Monte Carlo) V9是基于强化学习训练的AI，使用自对弈蒙特卡洛方法学习出牌策略。

9.1 模型架构

参数	值
网络类型	QNetworkV2
Dueling	False (不使用Dueling DQN)
隐藏层维度	512
动作嵌入维度	128
观测维度	764
动作空间	108维one-hot
设备	CPU

9.2 庄闲分离

DMC V9使用庄闲双网络设计：

- `banker_net`：庄家队使用的Q网络
- `xianjia_net`：闲家队使用的Q网络
- 推理时根据 `player_id in room.bankers` 选择对应网络

设计理由：庄家和闲家的目标函数不同（庄家要控分，闲家要抢分），分离网络让每个角色学到更专精的策略。

9.3 观测编码（764维）

```
obs = [hand_rep(108) + trick_rep(432) + played_rep(108) + color_rep(4) + level_rep(13)
      = 693 (base) + 71 (v11 extra) = 764
```

维度	大小	编码内容
hand_rep	108	手牌（108张牌的one-hot）
trick_rep	432	当前轮4人出牌（每人108维）
played_rep	108	已出的所有牌
color_rep	4	主花色（a/b/c/d四选一）
level_rep	13	当前级牌（A-K十三选一）
banker_rep	4	庄家座位
current_rep	4	当前玩家座位
score_rep	20	分数（闲家10桶 + 庄家10桶）
v11 extra	71	V11额外特征（详细牌力评估）

分数编码：- 闲家得分：`min(score_now / 20, 9)` → 第0-9桶 - 庄家得分：`min((100 - score_now) / 20, 9)` → 第10-19桶

9.4 动作编码

出牌动作编码为108维one-hot向量：

1. 通过 `_enumerate_legal_plays()` 枚举所有合法出牌组合
2. 每个组合转换为Card对象列表
3. 用 `cards_to_onehot()` 编码为108维向量
4. 去重（相同card_type组合视为同一动作）

9.5 推理流程

1. 构建观测 → `encode_obs_v11(room, player_id)` → 764维向量
2. 枚举合法动作 → `_enumerate_legal_plays()` → N个合法出牌
3. 编码动作 → `cards_to_onehot()` → N×108维矩阵
4. 选择网络 → `banker_net` 或 `xianjia_net`
5. 前向推理 → `net.evaluate_actions(obs, action_onehots)` → N个Q值
6. 选最优 → `argmax(Q值)` → 最优合法动作
7. 返回 `card_strs` 列表

9.6 单例与懒加载

DMCAI使用单例模式避免重复加载70MB模型：

```
class DMCAI:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

通过 `get_dmc_ai()` 全局接口懒加载，首次调用时才加载模型。

9.7 _GameWrapper适配器

DMC AI的训练环境 (`rl_shengji/env.py`) 与Web服务器的GameRoom接口不同，需要适配器：

```
class _GameWrapper:
    def __init__(self, room):
        self.room = room

    def get_state(self, player_id):
        # 从room构建693维base观测
        # 复用env.py中的编码逻辑
```

适配器将GameRoom实例包装为训练环境接口，使obs_v11编码器能直接使用。

10. AI决策优先级与Fallback机制

10.1 出牌决策三级Fallback

DMC V9 出牌 → 规则AI出牌 → 安全出牌(`safe_play_fallback`)

1. DMC V9 (首选) :
2. 调用 `dmc_decide_play(room, player_id)`
3. 返回card_strs列表，传入 `handle_play()` 验证合法性
4. 如果返回None或出牌非法 → fallback
5. 规则AI v28 (次选) :
6. 调用 `AI(player, ...).decide_play(epoch_cards, ...)`
7. 如果返回空或出牌非法 → fallback

8. 安全出牌（兜底）：

9. `_safe_play_fallback(player)` 方法

10. 首出：出最小单牌

11. 跟牌：按规则出最小合法牌

12. 保证一定能出牌

10.2 非出牌决策

亮主、扣牌、拾牌、换牌等决策不使用DMC，仍使用规则AI：

决策	AI模块	说明
亮主	规则AI.decide_liangzhu	DMC仅训练了出牌策略
扣牌	规则AI.decide_koupai	扣牌策略依赖规则推理
拾牌	规则AI.decide_shipai	拾牌逻辑简单
换牌	规则AI.decide_huanpai	换牌逻辑简单
出牌	DMC V9 → 规则AI → 安全出牌	三级fallback

10.3 DMC调用统计

系统内置DMC调用统计：

```
_dmc_stats = {
    'dmc_calls': 0,      # DMC总调用次数
    'dmc_success': 0,    # DMC成功次数
    'fallback_calls': 0  # Fallback到规则AI次数
}
```

可通过 `get_dmc_stats()` 查询。在50局测试中，DMC成功率 > 99.9%。

10.4 各AI实力对比

基于500局独立评估的结果：

AI	vs v28胜率	特点
DMC V9	67.6%	最强，庄家为核心瓶颈(59.8%)
MCTS V8	~55%	蒙特卡洛树搜索，0.2s/步

AI	vs v28胜率	特点
PPO V11joint	~53%	PPO联合训练
规则AI v28	50%(基线)	局部优化天花板

附录A：游戏状态JSON格式

服务器广播的state_update消息结构：

```
{
  "action": "state_update",
  "room_id": "DM001",
  "phase": "playing",
  "players": [
    {
      "seat": 0,
      "name": "玩家1",
      "card_count": 12,
      "is_robot": false,
      "player_level": 1
    }
  ],
  "current_turn": 2,
  "now_color": "b",
  "now_level": "A",
  "score_now": 45,
  "bankers": [0, 2],
  "epoch_cards": [...],
  "epoch_players": [...],
  "hand": [...],
  "hole_cards_count": 4
}
```

附录B：调试API

指定亮主手牌（仅调试用）：


```
POST /api/debug/seed-liangzhu
Body: {"room_id": "TEST", "seat": 0, "cards": ["A-13-a", "A-13-b", "w-14-z"]}
```

附录C：Docker部署

```
# 构建镜像（CPU版torch, 约1.46GB）
docker build -t trump-game-server .

# 运行（挂载模型文件）
docker run -d -p 9999:9999 \
    -v /path/to/models_v9:/app/rl_dmc/models_v9:ro \
    trump-game-server

# 运行测试
docker run --rm \
    -v /path/to/models_v9:/app/rl_dmc/models_v9:ro \
    trump-game-server pytest test_dmc_unit.py test_dmc_e2e.py -v
```

附录D：v2.0更新日志（2026-06-10）

跟牌策略6项修改

#	修改项	说明
1	跟副单绝门AI策略	绝门时队友大贴分牌/对手大毙牌（出最小主牌）
2	跟副对1张补牌	去掉验证层强制补牌要求，AI优先副牌补，不够再用主牌
3	跟副对绝门	去掉验证层强制出主对，AI策略决定：场上有分>10才用主对毙牌
4	跟副连对绝门	去掉验证层限制，AI优先出无分副散牌，有主连对不主动毙牌
5	跟主连对独立分支	有主牌必须出所有主牌+副牌补齐，只有0张主牌才纯副牌
6	强检验	出牌数相等+手牌数相等assert+出牌数量不超过手牌（Counter检查）

Bug修复

- 出牌数量验证: `handle_play` 中用 `Counter` 计数比较替代简单 `in` 判断, 防止出2张同名牌但手中只有1张导致手牌数不等
- GAME_OVER保护: `auto_play_current_robot` 开头加 `if self.phase == GAME_OVER:`
`return []` 防止游戏结束后死循环
- 绝门判定统一: 验证层绝门判定统一为 `hand_color_count == 0` (非 `< 2`)
- 80-119分过庄Bug: 闲家夺庄+0级, 闲家胜 (非庄家胜)